

Matrix Embedding with LSBM and Hamming Codes

Marcel Auer

June 6, 2026

1. True Embedding with LSBM and Hamming Codes

Approach

- Implemented matrix embedding explicitly by generating parity-check matrices H for Hamming codes.
- **Syndrome Calculation:** Compute $s = H \cdot y \pmod{2}$. The required change is $m - s \pmod{2}$.
- **Modification:** Find the index to change based on the difference.
- **LSBM:** In LSB Matching, changing the LSB equates to randomly adding or subtracting 1 from the pixel intensity, ensuring boundaries (like 0 and x_{max}) are respected.

2. Simulating Hamming Codes

Simulation Strategy

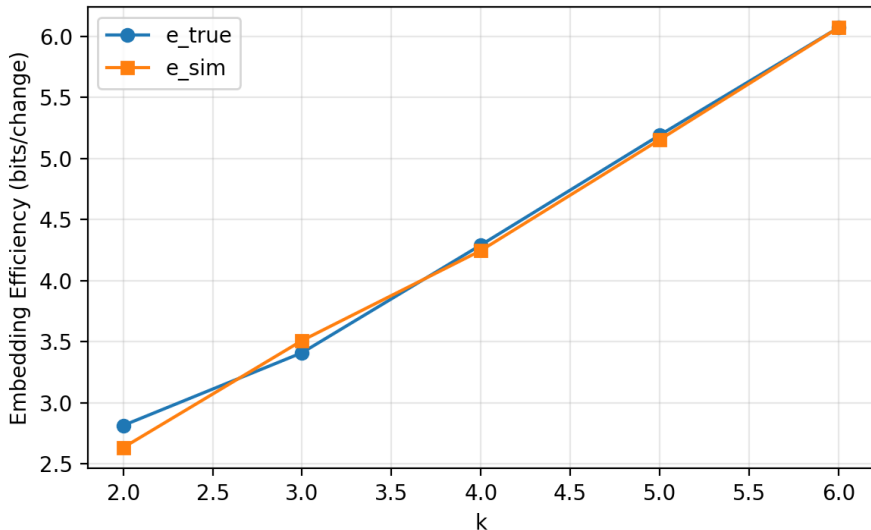
- Each Hamming code block changes with probability $P(\text{change}) = \frac{2^k - 1}{2^k}$.
- The total number of modified blocks can be sampled directly using a Binomial distribution:
 $\sim \text{Binomial}\left(N_{\text{blocks}}, \frac{2^k - 1}{2^k}\right)$.
- Solves the need to explicitly execute code matrices over covers, massively increasing simulation speed.

Interpolation for Target α

- To achieve non-integer embedding rates α , we linearly interpolate the change probabilities per element of the two neighboring Hamming codes k and $k + 1$.

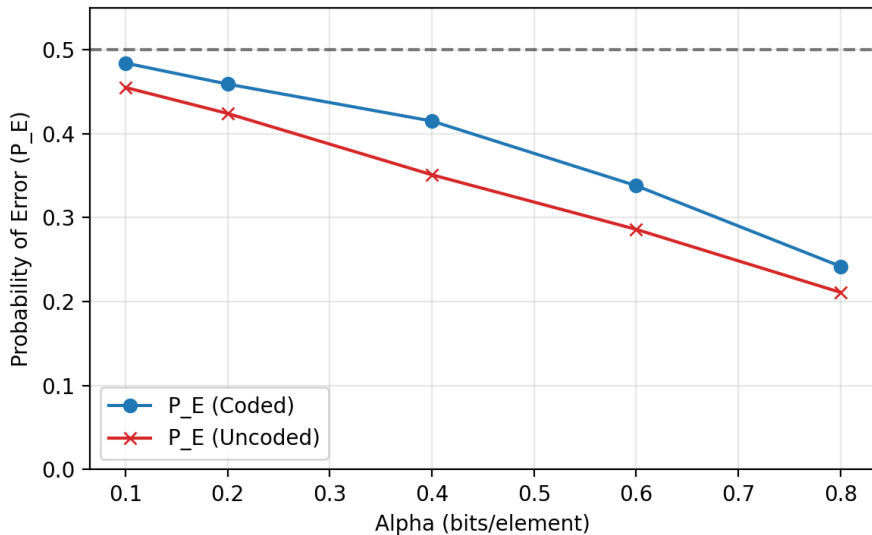
Embedding efficiency ε for different k

True vs Simulated Efficiency



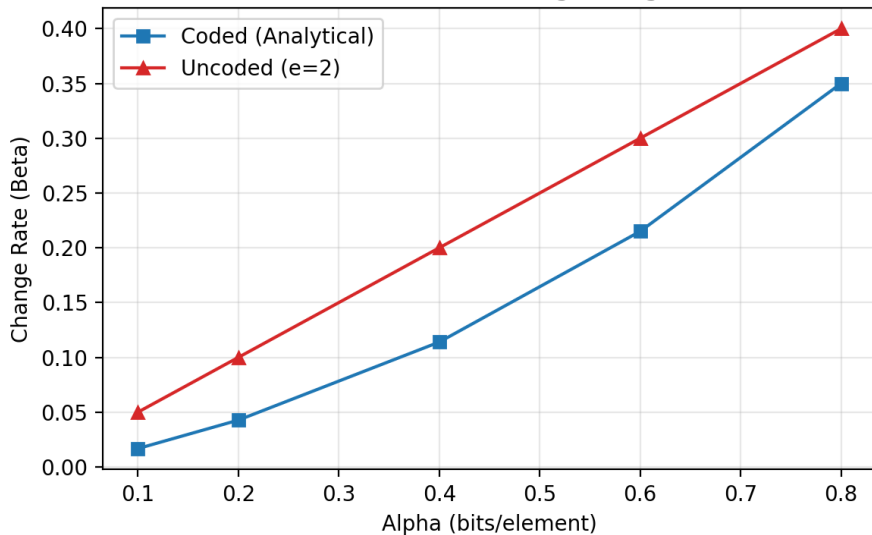
LSBM hamming against uncoded LSBM simulations

Simulated HCF-COM Attack Performance

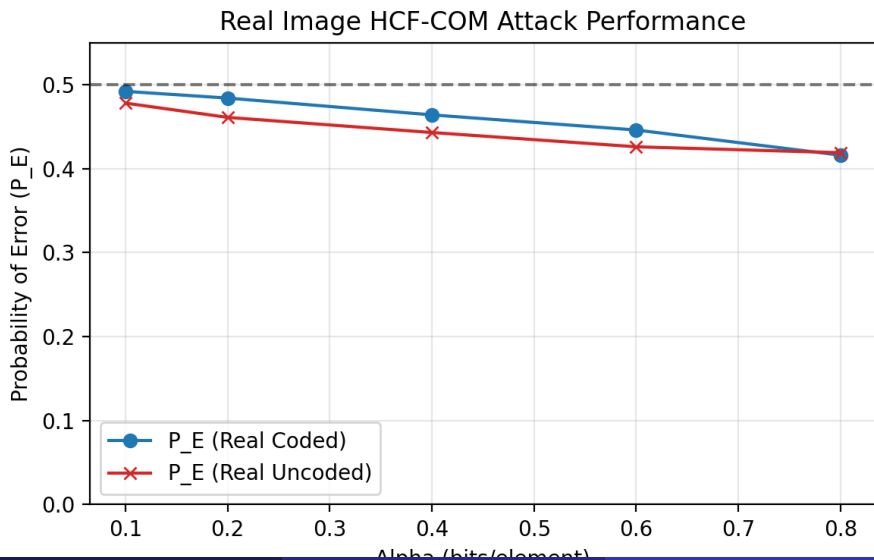


Change rate β for different k

Simulated Embedding Changes



LSBM hamming against uncoded LSBM 500 grayscale images



Summary

- **Completed Implementation:** Fully functional LSB Matching based upon matrix embedding with Hamming codes.
- **Fast Simulator derived:** Allows generating theoretical impacts for arbitrary embedding rates α via local interpolation.
- **Results:** Hamming codes reduce the change rate β therefore improve detectability against HCF.